

A New Reducer for Indexed Concatenation

`concat.py` compared with `reduce.py`, its predecessor in the *musIC* workbench

musIC project · October 5, 2026 · Code: `2026/musicic/musicic/core/concat.py` (new, 550 lines) and `core/reduce.py` (previous, 381 lines, with `core/findseq.py`, 173 lines)

1 Summary

Both programs take a list of integer rows (for example `{{1}, {1, 2}, {2}, {2}, {1, 1}, ...}`) and try to rewrite it as nested indexed concatenations (ICs), so that the expression expands back to exactly the original list. `reduce.py` is a Python port of the strategy in Mathematica's `ReduceSetList` (package `SSSiCv102.wl`). `concat.py` is a new reducer written from scratch, with a different search strategy.

The measure throughout is **leaves**: the number of rows you have to write down to state an expression. A plain row counts 1, and an IC counts the leaves of its body. A list that only has its repeated rows collapsed is the baseline.

Reference set	Rows	Repeats collapsed	Mathematica <code>ReduceSetList</code>	<code>reduce.py</code>	<code>concat.py</code>
Set 2 (first 463 rows)	463	110	7	110	7
Set 3	943	202	13	202	8
Set 4	743	270	6	270	6
Set 5	499	121	120	121	14
Set 6	499	145	141	145	16

Leaves in each result; lower is better. Every result in the table expands back to its input exactly. The Mathematica column was produced by running `ReduceSetList` from `SSSiCv102.wl` through `wolframscript`.

Main findings. On these five sets, `reduce.py` never finds an indexed block. It only collapses repeated rows. `concat.py` finds the main block in every set and matches or beats Mathematica on all five: equal on Sets 2 and 4, and smaller on Set 3 (8 against 13) and on Sets 5 and 6 (14 against 120, 16 against 141), where `ReduceSetList` also finds no main block.

The difference comes mostly from two design decisions, explained in Section 2: `concat.py` treats the repeat count of a run as data to be fitted, and it scores every candidate block instead of committing to the first one that matches.

2 Key differences in the code and the reasoning

Both programs have the same overall shape: find stretches of the list that repeat with a regular change, fit a formula for each changing number as a function of an index k , and accept a candidate only if it expands back exactly. They differ in what counts as "the same shape", which numbers are fitted, how a block is chosen and extended, and what happens to the rows around it.

Question	reduce.py (port of ReduceSetList)	concat.py (new)
What has to match between periods?	A skeleton in which every integer except 1 becomes 0. A run is <code>IC(...)</code> and a single row is <code>{...}</code> , so the two never match.	Only the row lengths. A single row, a run and a progression are all one kind of object, an <i>atom</i> (start row, step, count).
Which numbers are fitted as functions of k ?	The numbers inside rows. A run's repeat count is not read.	The numbers inside rows, the step of a progression, and the repeat count .
How is a block chosen?	The stretch with the most identical skeletons in a row. It is then fitted, and the round fails if the fit fails.	The block is grown one period at a time while every column can still be fitted. Every candidate is scored by leaves saved, and the best one wins.
How is a block extended?	Only to the right (<code>_absorb_right</code>).	Both directions, by evaluating the formulas at $k = 0, -1, \dots$ and checking them against the raw rows (counts of 0 and 1 allowed), at every rotation of the body.
What happens to the rows around a block?	The whole item list is passed through again, already partly reduced.	The rows before and after are reduced again from the raw rows, independently.
Which formulas?	<code>findseq.fit</code> : constant, polynomial, geometric, polynomial $\times$ geometric, order-2 recurrence. A line through only two points is refused.	Constant, linear, $a + c \cdot b^k$, quadratic, $a + bk + c \cdot b^k$. A line through only two points is allowed only as a constant or a step of ± 1 .

2.1 Run counts are data, not part of the shape

Repeat counts that change from period to period are the most common pattern in the reference sets: "the row `{1, k+2}`, written $k-1$ times". `reduce.py` cannot express this, because when it collects the numbers to fit it reads only the body of a run, never its count:

```
# reduce.py
def _numbers(item):
    if isinstance(item, IC):
        return [v for a in item.args for v in _numbers(a)]
    # item.it.stop, the repeat count, is never read
    return list(item)
```

The fitted template then reuses the first period's count for every period, the expansion check fails, and the block is thrown away. Tested directly on three runs, $\{5\} \times 2$, $\{6\} \times 3$, $\{7\} \times 4$: `reduce.py`'s `_fit_block` returns `None`. With the counts held equal it returns `€(n1..3)[€(2)[{n1 + 4}]]`. So it handles a run whose count stays the same, and fails as soon as the count changes.

`concat.py` represents every run, single row and arithmetic progression as one kind of object, and feeds the count into its own column like any other number:

```
# concat.py
@dataclass(frozen=True)
class Atom:
    a: Row          # first row
    d: Row | None    # step between rows; None for a single row (a wildcard)
    n: int           # how many rows

def _feed(cols, atom, i, k):
    ...                # every entry of a, then of d (if n > 1), then:
    return cn[i].add(k, atom.n)  # the count is a column like the others
```

Because a single row has no step, its step is a *wildcard*: it matches any step. That lets a period where something occurs once line up with periods where the same thing is a run or a progression.

2.2 The skeleton: the value 1 and the run/row distinction

`reduce.py` keeps `ReduceSetList`'s rule of turning every integer except 1 into 0 before comparing periods. So `{1, 1}` and `{2, 2}` have different skeletons (`{1,1}` against `{0,0}`). In `ReduceSetList` this keeps marker rows such as `{1}` and `{1, 1, 2}` aligned. It also breaks any period where a varying number happens to equal 1, which is common at $k = 1$. `concat.py` compares row lengths only and lets the formula fit decide whether the values agree. The cost is more candidate matches, which the scoring step has to sort out.

2.3 Choosing a block: score everything instead of committing early

`reduce.py` picks the stretch with the most consecutive identical skeletons, then tries to fit it. If the fit fails, nothing is reduced in that round. `concat.py` starts from every position and period length and grows each candidate one period at a time while all its columns still fit. The growth uses a *loose* test: a formula that needs one more point to be confirmed does not stop growth. Each candidate is then confirmed strictly and shortened if needed. Finally, the candidate that saves the most leaves wins. When two save the same, the one with simpler formulas wins:

```
# concat.py, _best_block (abridged)
for _, s, p, q in _candidates(atoms)[:EXTEND_TOP]:
    while q >= 2 and (body := _body(atoms, s, p, q)) is None:
        q -= 1                # the last period was only loosely fitted
    for r in range(p):        # every rotation: where does a period start?
        lo, hi, r0, r1 = _extend(rows, body, r, ...)
        saving = _rle_leaves(rows[r0:r1]) - p
        cand = Block(r0, r1, _rebase(body, r, lo, hi), hi - lo + 1, saving)
        if best is None or cand.rank > best.rank:  # rank = (saving, -formula cost)
            best = cand
```

The loose/strict split matters for formulas such as $2^k + k$, which need three points to determine and a fourth to confirm. A purely strict search stops at the third period, before the confirming point arrives.

2.4 Extending a block at the row level, in place of "rolling"

The atom grouping cannot line up a period in which a run has count 0 (the run is absent) or count 1 (a bare row), and these are usually the first one or two periods. `ReduceSetList` recovers them by "rolling" the block left one item at a time. `reduce.py` has no left extension, only `_absorb_right`. `concat.py` evaluates its fitted

body directly at $k = 0, -1, \dots$ and at $k = q+1, \dots$, and keeps every period that reproduces the neighbouring rows exactly. A count of 0 simply produces no rows. The same is done for each rotation of the body, which settles where a period starts. Then the index is shifted so the block always runs from $k = 1$:

```
# concat.py
def _extend(rows, body, r, lo, hi, r0, r1):
    for _ in range(len(rows)):
        got = _period_rows(body, r, lo - 1)          # the body evaluated one period earlier
        if not got or rows[r0 - len(got):r0] != got:
            break
        lo, r0 = lo - 1, r0 - len(got)
    ...                                              # and the same to the right
```

2.5 The rows on either side are reduced from scratch

`reduce.py`, like `ReduceSetList`, keeps working on one list that becomes more and more reduced, so a small block made early can sit across the boundary of a larger one and block it. In Mathematica's result for Set 3, the main block stops one period short because $\{1, 3\}, \{1, 4\}$ had already been turned into $\epsilon^2[\{1, 2+n\}]$. A step-by-step trace of the Wolfram code confirmed this. `concat.py` chooses the best block on the raw rows, then calls itself on the raw rows before and after it:

```
# concat.py
def _reduce(rows):
    block = _best_block(rows)
    if block is None:
        return _rle_tree(rows)
    return _reduce(rows[:block.r0]) + [_block_tree(block)] + _reduce(rows[block.r1:])
```

2.6 Which formulas to trust

Any two points lie on a straight line, so a line fitted to two periods proves nothing. `reduce.py` refuses such fits outright. Mathematica accepts them, and the notebooks rely on fits like $4 - n$. `concat.py` takes a middle position: a line through two points is accepted only if it is constant or steps by ± 1 . Every richer formula must also predict a point it was not fitted on. An early version without this rule produced $\epsilon(k..2)[\epsilon(2k-1)[\{59 - 29k, k + 29\}]]$: it was valid and saved one leaf, but it described nothing real. With the rule in place, those rows are left as written (example F below).

2.7 What stays the same: nothing is returned unverified

Both programs check the final expression by expanding it and comparing it with the input. If the check fails, they fall back to collapsing repeated rows only. So any extra compression from `concat.py` never risks a wrong answer: every claim of compression in this report comes from an expression that was expanded and compared with the input.

3 How the two programs concatenate differently

Six small constructed lists, each isolating one of the differences above, run through both programs. $\epsilon(n)[x]$ repeats x n times; $\epsilon(k..N)[...]$ evaluates the body for $k = 1, \dots, N$.

A. Repeat counts that grow with k (Section 2.1) reduce.py 5 leaves concat.py 1 leaf

Input $\{5\} \times 2, \{6\} \times 3, \{7\} \times 4, \{8\} \times 5, \{9\} \times 6$ (20 rows)

reduce.py $\{\epsilon(2)[\{5\}], \epsilon(3)[\{6\}], \epsilon(4)[\{7\}], \epsilon(5)[\{8\}], \epsilon(6)[\{9\}]\}$

concat.py $\{\epsilon(k..5)[\epsilon(k+1)[\{k+4\}]]\}$

B. A count of 1 next to longer runs, and the value 1 in the skeleton (2.1, 2.2) 12 2

Input $\{1,1\}, \{1\}, \{2,2\}, \{1\} \times 2, \{3,3\}, \{1\} \times 3, \dots \{6,6\}, \{1\} \times 6$ (27 rows)

reduce.py $\{\{1, 1\}, \{1\}, \{2, 2\}, \epsilon(2)[\{1\}], \{3, 3\}, \epsilon(3)[\{1\}], \{4, 4\}, \epsilon(4)[\{1\}], \{5, 5\}, \epsilon(5)[\{1\}], \{6, 6\}, \epsilon(6)[\{1\}]\}$

concat.py $\{\epsilon(k..6)[\{k, k\}, \epsilon(k)[\{1\}]]\}$

C. A first period with a run of length 0, the Set 2 pattern (2.4) 17 3

Input $\{2\}, \{1,2,3\}, \{3\}, \{1,1,4\}, \{1,2,4\}, \{4\}, \{1,1,5\} \times 2, \{1,2,5\}, \dots$ (27 rows)

reduce.py $\{\{2\}, \{1, 2, 3\}, \{3\}, \{1, 1, 4\}, \{1, 2, 4\}, \{4\}, \epsilon(2)[\{1, 1, 5\}], \{1, 2, 5\}, \{5\}, \epsilon(3)[\{1, 1, 6\}], \dots\}$

concat.py $\{\epsilon(k..6)[\{k+1\}, \epsilon(k-1)[\{1, 1, k+2\}], \{1, 2, k+2\}]\}$

At $k = 1$ the middle item has count 0. The atom grouping cannot line that period up; the row-level extension recovers it.

D. A small pattern sitting across the boundary of a large one, the Set 3 pattern (2.5) 46 8

Input $\{1,1,2\}, \{\}, \{1,3\}, \{1,1,2\}, \{\}, \{1,2\}, \{3,4\}, \{1,1,2\}, \{\}, \{1,3\}, \{1,4\}, \{4,5\}, \dots$ (74 rows)

reduce.py no block found: 46 leaves, repeated rows collapsed only

concat.py $\{\epsilon(k..2)[\{1, 1, 2\}, \{\}, \{1, 4-k\}], \epsilon(k..8)[\{k+2, k+3\}, \{1, 1, 2\}, \{\}, \{1, k+2\}, \epsilon(k)[\{1, k+3\}]]\}$

The main block covers the period starting at $\{3, 4\}$, which Mathematica leaves outside its block, and the head is reduced separately into a two-period block.

E. An inner progression whose length and values grow as 2^k , the Set 4 pattern 38 4

Input $\{1,2,3\}, \{3\} \times 3, \{1\}, \{1,2,3\}, \{3,4,5\}, \{5\} \times 5, \{1\}, \{1,4,5\}, \{1,5,6\}, \{1,6,7\}, \dots$ (68 rows)

reduce.py no block found: 38 leaves

concat.py $\{\epsilon(k..4)[\{2^k-1, 2^k, 2^k+1\}, \epsilon(2^k+1)[\{2^k+1\}], \{1\}, \epsilon(j..2^k-1)[\{1, 2^k+j-1, 2^k+j\}]]\}$

This is the same expression Mathematica produces for the full Set 4.

F. No real pattern: both programs should leave it alone (2.6) 4 4

Input $\{30,30\}, \{1,31\} \times 3, \{5\}, \{2,9\}$

both $\{\{30, 30\}, \epsilon(3)[\{1, 31\}], \{5\}, \{2, 9\}\}$

Without the two-point rule, `concat.py` would fit 59 – 29k here.

3.1 Where both programs succeed

For balance: on simple, regular lists both programs find the structure. `concat.py` is never worse on the cases tested, and is sometimes smaller because it nests a progression inside a block:

Input	reduce.py	concat.py
$\{16, 60+2i\}, i = 0..7$	$\epsilon(n1..8)[\{16, 2 \cdot n1 + 58\}]$ (1)	$\epsilon(k..8)[\{16, 2k + 58\}]$ (1)
$\{8,61\}, \{8,62\}, \{8,63\}, \{16,67\}$ four times	$\epsilon(4)[\{8, 61\}, \{8, 62\}, \{8, 63\}, \{16, 67\}]$ (4)	$\epsilon(k..4)[\epsilon(j..3)[\{8, j + 60\}], \{16, 67\}]$ (2)
$\{k, 2k+1\}$ twice each, $k = 1..8$	$\epsilon(2)[\{1, 3\}], \epsilon(n1..7)[\epsilon(2)[\{n1+1, 2n1+3\}]]$ (2)	$\epsilon(k..8)[\epsilon(2)[\{k, 2k + 1\}]]$ (1)

4 Why this is an upgrade, and what it costs

REDUCE.PY

Finds blocks only when every period has exactly the same shape, including every run having the same repeat count. Real sets from this project vary their repeat counts from period to period almost every time, so on all five reference sets it falls back to collapsing repeated rows.

CONCAT.PY

Finds blocks whose repeat counts, steps and values all change with k , including periods where something is missing or occurs once. It recovers the main block in every reference set, does as well as or better than Mathematica on all five, and stays lossless by the same expansion check.

Costs and limits, stated plainly:

- **Speed.** `concat.py` takes 3–11 s on the reference sets (463–943 rows); `reduce.py` takes under 0.1 s, largely because it finds nothing to fit. Mathematica took under 3 s on Sets 2–4, and about 4.5 minutes each on Sets 5 and 6.
- **Nesting depth.** Blocks nest two levels: an outer block whose body holds rows, runs and arithmetic progressions. A body that contains a repeating group of several rows with its own index is not yet searched for.
- **Formula family.** The formulas are those listed in Section 2, not everything Mathematica's `FindSequenceFunction` can find.
- **No optimality guarantee.** Choosing the best block at each step, then reducing each side, is a heuristic. It is not a search for the smallest possible expression. The tie-break on "simpler formulas" is a readability judgement.
- **Edge periods.** As agreed for this work, a partial period at the start or end of a set is left as plain rows and runs, not forced into the block.

5 How the results were checked

- **Exact expansion.** Every result in this report was expanded and compared with its input. The test suite (`tests.py`, section 7b) checks this for all five reference sets and also fails if any result gets larger than the sizes above.
- **Mathematica.** The `IndexedConcatenate[...]` form of each `concat.py` result for Sets 2–6 was loaded into Mathematica with `SSSiCv102.wl`. `ExpandAll` reproduced each original set exactly.
- **Reproducing the baseline.** `ReduceSetList` was run through `wolframscript` on the same rows. On Sets 2–4 it reproduced the expressions recorded in the original notebooks exactly, including the index names.

- **Random tests.** 300 lists: 150 random, and 150 generated from a random period with constant, linear, k and 2^k counts plus random rows at both ends. No result failed to expand back, and none was larger than collapsing repeated rows. Of 100 generated lists in a separate run, 98 reduced to 12 leaves or fewer. The 2 others (32 and 36 rows) have not been diagnosed yet.

A Appendix: `concat.py` output on the reference sets

Each expression expands to the listed number of rows exactly.

Set 2 (463 rows, 7 leaves)

$\{\{1\}, \{1, 2\}, \epsilon(k..27)[\epsilon(2)[\{k + 1\}], \{1, 1\}, \epsilon(k - 1)[\{1, 1, k + 2\}], \{1, 2, k + 2\}], \epsilon(2)[\{29\}]\}$

Set 3 (943 rows, 8 leaves)

$\{\{1, 1, 2\}, \{\}, \{1, 3\}, \epsilon(k..40)[\{1, 1, 2\}, \{\}, \{1, k + 1\}, \epsilon(k - 1)[\{1, k + 2\}], \{k + 2, k + 3\}]\}$

Set 4 (743 rows, 6 leaves; the same expression as Mathematica's)

$\{\epsilon(k..7)[\{2^k - 1, 2^k, 2^k + 1\}, \epsilon(2^k + 1)[\{2^k + 1\}], \{1\}], \epsilon(j..2^k - 1)[\{1, 2^k + j - 1, 2^k + j\}]\},$
 $\{255, 256, 257\}, \epsilon(220)[\{257\}]\}$

Set 5 (499 rows, 14 leaves)

$\{\{1, 1, 4\}, \{1, 1\}, \{1, 1, 2\}, \epsilon(k..28)[\{k + 1, k + 1\}, \epsilon(k - 1)[\{1, k + 2\}], \{k + 2, k + 3\}], \{1, 1, 2\}],$
 $\{30, 30\}, \epsilon(3)[\{1, 31\}], \epsilon(25)[\{1\}], \{\}, \{1, 1, 2\}, \{\}, \epsilon(2)[\{1\}]\}$

Set 6 (499 rows, 16 leaves)

$\{\{1, 1\}, \{1\}, \{1, 3\}, \{1, 1\}, \{1\}, \epsilon(k..27)[\{1, k + 1\}, \epsilon(k - 1)[\{1, k + 2\}], \{k + 2, k + 3\}], \{1, 1\}, \{1\}],$
 $\{1, 29\}, \epsilon(5)[\{1, 30\}], \epsilon(22)[\{1\}], \{\}, \{1, 1\}, \epsilon(5)[\{1\}]\}$

For comparison, Mathematica's `ReduceSetList` on Set 3 gives (13 leaves):

$\{\epsilon^2[\{1,1,2\}], \{\}, \{1, 4-n\$2\}], \{3,4\}, \{1,1,2\}, \{\}, \epsilon^2[\{1, 2+n\$1\}], \epsilon^{38}[\{3+n\$3, 4+n\$3\}], \{1,1,2\},$
 $\{\}, \{1, 3+n\$3\}, \epsilon^{1+n\$3}[\{1, 4+n\$3\}]\}, \{42,43\}\}$

In the musIC app, the **Concatenate** page (`/concatenate`) runs `concat.py` on any pasted set and loads these reference sets for comparison.